Inteligencia Artificial

Episodio VIII
Sistemas basados en conocimiento



Dr. ALEJANDRO JAVIER GARCÍA http://cs.uns.edu.ar/~ajg

DEPARTAMENTO DE CIENCIAS E INGENIERÍA DE LA COMPUTACIÓN - UNIVERSIDAD NACIONAL DEL SUR - BAHÍA BLANCA - ARGENTINA

(Algunas) áreas de Inteligencia Artificial

Resolución automática de problemas

Sistema Basados en Conocimiento

Aprendizaje automático (Machine Learning)

Procesamiento de Lenguaje Humano

Razonamiento Automático

- Dinámica del Conocimiento.
- Manejo de Información incompleta
- Manejo de contradicciones
- Razonamiento bajo Incertidumbre.
- Razonamiento basado en informantes.
- Razonamiento basado en argumentos.
- …

Sistemas Multiagente

Representación de Conocimiento

Agentes inteligentes

Sistemas para toma de decisiones

Sistemas de Recomendación

Planificación automática

Robótica



"Es la Ciencia e Ingeniería de construir artefactos inteligentes, en especial programas inteligentes de computadora. También se relaciona con el uso de computadoras para entender la inteligencia humana".

Aplicaciones con razonamiento automático

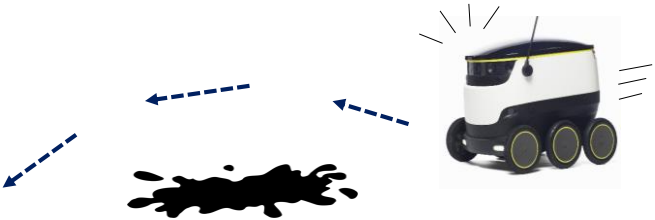
Existe una gran variedad de aplicaciones en donde los agentes como parte de sus tareas deben:

- tomar una decisión
- dar una recomendación
- sacar una conclusión

y esto deben hacerlo utilizando la información que es percibida por los agentes en el momento que están realizando sus tareas.

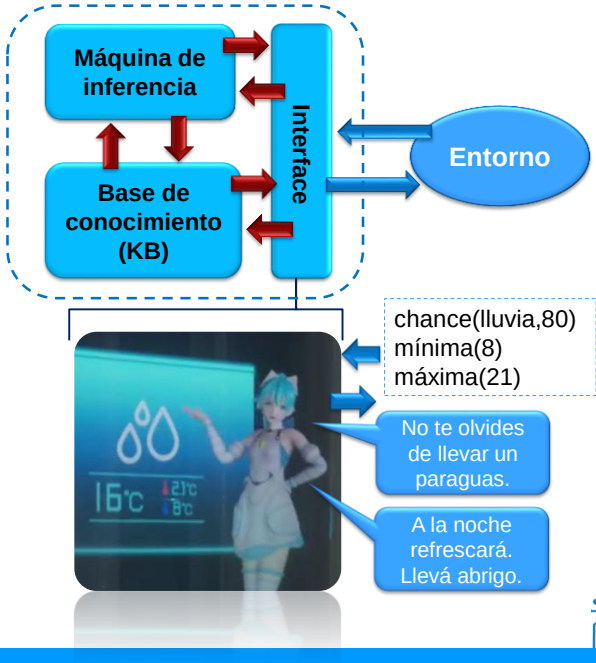
Por lo tanto, las conclusiones, recomendaciones o decisiones dependerán tanto de la información que tengan en ese momento, como de las reglas que dispongan para razonar.





Sistema Basado en Conocimiento (SBC)

Arquitectura básica de un Sistema Basado en Conocimiento:

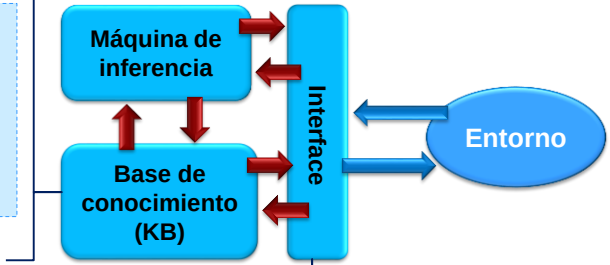


Arquitectura de un sistema: Es un modelo conceptual que define la estructura del sistema, indicando (generalmente en forma gráfica) los elementos que lo componen y como se relacionan.

Para disponer de sistemas con razonamiento automático son necesarias herramientas tanto para representar conocimiento del dominio, como para obtener conclusiones.

Ejemplo simplificado de SBC del agente de compañía “Okaeri”.

chance(lluvia, X) y X > 50 → recomendar(paraguas)
mínima(Temp) y Temp < 10 → recomendar(abrigo)
verano → recomendar(sombrero)
verano → recomendar(protector solar)
hemisferio(sur) y (mes(12) o mes(1) o mes(2)) → verano
hemisferio(norte) y (mes(6) o mes(7) o mes(8)) → verano
…

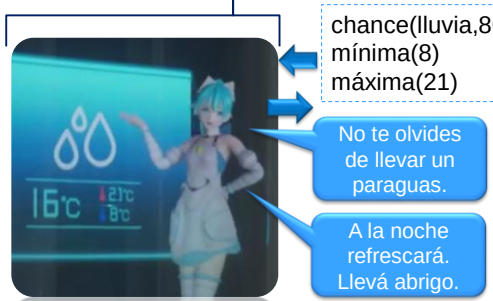


Este ejemplo insinúa algunas reglas para hacer inferencias qué podrían ser parte de la base de conocimiento de Okaeri.

La máquina de inferencia del SBC usará esas reglas para sacar conclusiones a partir de los datos percibidos.

Este diseño modular permite agregar dinámicamente reglas, sin cambiar código del mecanismo de inferencia.

La interface se encarga de mostrar adecuadamente las sugerencias, procesar las percepciones, buscar información en la web, interpretar comandos de voz de los humanos, etc.



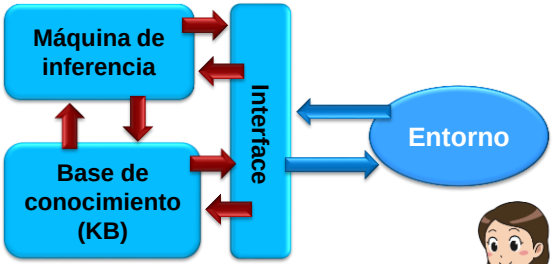
¿Cómo se implementa un SBC?

Cómo ya sabés, tener una sistema modular nos da muchas ventajas, las cuales vamos a aprovechar.

La interface depende mucho del dominio de aplicación, y también de los sensores y acciones disponibles para el agente.

En lo que sigue, me voy a concentrar alternadamente en cada uno de los módulos, te voy a mostrar diferentes máquinas de inferencia, distintas formas de representar conocimiento, y algunas formas de implementar la interface.

Claramente, la representación del conocimiento debe estar en completa sintonía con el mecanismo de inferencia.

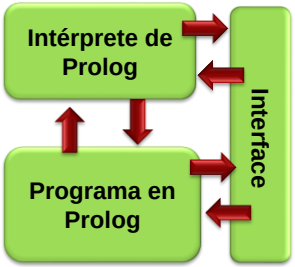


¿Puedo usar una máquina de inferencia ya existente?

Por supuesto. Por ejemplo, un intérprete de Prolog.

Si usas Prolog como mecanismo de inferencia, entonces la base de conocimiento, para estar en sintonía, debe estar representada con un programa Prolog.

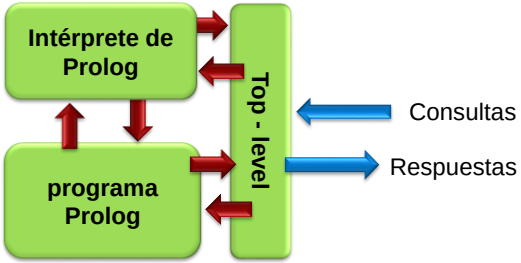
A continuación veremos un ejemplo implementado en Prolog.



Si representamos el conocimiento con un programa en Prolog, podremos usar el intérprete de Prolog como la máquina de inferencia del SBC.

SBC que usa la máquina de inferencia de Prolog

```
recomendar(paraguas):- chance(lluvia, X), X > 50.
recomendar(abrigo) :- minima(Temp), Temp < 10.
recomendar(sombrero):- verano.
recomendar(protectorsolar):- verano.
verano :- hemisferio(sur), mes(M), member(M,[12,1,2]).
verano :- hemisferio(norte), mes(M), member(M,[6,7,8]).
recordar(cumple(P)) :- agenda(hoy, cumple, P).
chance(lluvia,80).
minima(8).
```




```
?- recomendar(X).
   X = paraguas;
   X = abrigo
?- recordar(Algo).
no.
```



La base de conocimiento en este caso será un programa Prolog con reglas y hechos. Los hechos resaltados en color pueden obtenerse dinámicamente de internet, y estar actualizados.

Observá que el conocimiento es un módulo independiente del mecanismo de inferencia, lo cual nos da varias ventajas. Por ejemplo: que se agreguen o quiten reglas o hechos de manera dinámica (forma simple de adaptarse a los cambios), o también para usarlo para diferentes dominios de aplicación.

Como una interface rudimentaria podés usar el top-level del interprete de Prolog, hacer consultas y obtener respuestas.

Cuestiones que hay que abordar en SBC

En los SBC hay que tener en cuenta cómo resolver las siguientes cuestiones:

- Derivación de nuevo conocimiento.

  Si hay una conexión entre dos o más piezas de conocimiento, esa conexión debería poder aprovecharse para deducir, hacer diagnósticos, encontrar explicaciones, etc.

- Explicabilidad.

  Las razones que llevan a una conclusión o decisión deberían poder auditarse.

- Manejo de información incompleta.

  La información que dispone un agente es en general parcial.

- Manejo de la dinámica de la información.

  La información generalmente cambia con el tiempo o el agente la busca cuando la necesita.

- Manejo de información contradictoria.

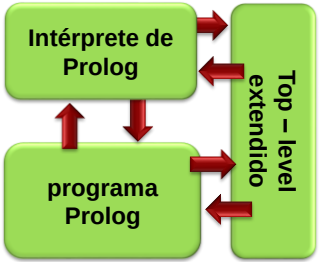
  Al obtener información puede haber contradicciones en el tiempo o de distintos puntos de vista.



¿Por qué son importantes cada una de las cuestiones?

SBC que usa la máquina de inferencia de Prolog

```
recomendar(paraguas):- chance(lluvia, X), X > 50.
recomendar(abrigo) :- minima(Temp), Temp < 10.
recomendar(sombrero):- verano.
recomendar(protectorsolar):- verano.
verano :- hemisferio(sur), mes(M), member(M,[12,1,2]).
verano :- hemisferio(norte), mes(M), member(M,[6,7,8]).
recordar(cumple(P)) :- agenda(hoy, cumple, P).
chance(lluvia,80).
minima(8).
```





Ale: recomendame.
Oki: paraguas y abrigo.
Ale: explicame.
Oki: chance de lluvia es 80%
temperatura mínima de hoy 8 grados
Oki: Hoy no tenés reuniones agendadas.

Con poco esfuerzo de programación podés mejorar la interface y hacer un “top-level extendido” con una interacción más elaborada. Como el ejemplo de la derecha con el agente “Oki”.

También, con poco esfuerzo podés usar las derivaciones que hace Prolog para cada consulta, y dar una explicación de cómo se llegó a la conclusión para cada respuesta.

El agente también podría ser proactivo y dar sugerencias o recordatorios aunque no se realicen consultas.

Negación por falla en Prolog

Es normal que en un SBC quieras expresar información negada.

En Prolog existe una manera muy particular de expresar la negación, llamada negación por falla usando el operador **\+** (formado por el símbolo **** en Prolog usado para negar algo y el **+** que significa derivable) [ver más].

La razón de su nombre es que, desde el punto de vista operacional, dado un predicado o consulta **p**, la derivación de **\+p** tiene éxito cuando la derivación de **p** falla, y **\+p** falla cuando la derivación de **p** tiene éxito (ver ejemplo).

La negación por falla puede resultar muy útil para trabajar con información incompleta.

El operador \+ solamente puede ser usado para anteponer a un predicado en el cuerpo de una regla, o en una consulta.

Es importante destacar que dada sus características, no está permitido usar \+ en la cabeza de una regla, o en un hecho.

Incorrecto en Prolog

```
\+ abierto(C) :- cerrado(C).
\+ abierta(universidad).
```



```
inofensivo(koala).
animal(koala).
animal(wallaby).
peligroso(X):- animal(X), \+ inofensivo(X).
culpable(acusado2).
inocente(P) :- \+ culpable(P).
```

```
?- culpable(acusado2).
true.
?- inocente(acusado2).
false.
?- inocente(acusado1).
true.
?- culpable(acusado1).
false.
?- peligroso(wallaby).
true.
?- \+ inofensivo(emu).
true.
?- \+ peligroso(wallaby).
false.
```

Construyendo un agente usando un Sistema Basado en Conocimiento

Agente de recomendación en el Mercado de Valores (Mer-val)

Usando un SBC, te propongo hacer un agente de recomendación para el mercado bursátil, que pueda responder consultas sobre la compra de acciones de una determinada empresa, utilizando Prolog como máquina de inferencia. (1)

Me voy a enfocar solamente sobre la base de conocimiento, la cual contendrá reglas con conocimiento recolectado de expertos en el mercado bursátil, y además, hechos con información que pueda obtener de manera automática de sitios especializados en este tema.

El agente tendrá este comportamiento básico y la interface será el top-level de Prolog.

Comportamiento del agente de recomendación Mer-val

REPETIR

- esperar por una consulta
- actualizar la información de empresas con sitios especializados de Internet
- realizar inferencias con las reglas de los expertos en mercado bursátil
- dar la respuesta con la recomendación sobre la consulta

HASTA el fin



(1) La base de conocimiento que se mostrará a continuación, es una adaptación de un ejemplo desarrollado en *A. J. Garcia. Tesis de Doctor en Ciencias de la Computación.* 2000 UNS.

Representación de conocimiento en el domino Mer-val

A continuación, te voy a mostrar como está formada la base de conocimiento del agente Mer-val la cual contendrá diferente tipo de información, que se usará para responder consultas.



Información recopilada de expertos del mercado de valores: [2]

- Si las acciones de la empresa X están a buen precio, entonces hay una buena razón para comprarlas.
- Generalmente las acciones están a buen precio si así lo indican páginas especializadas en la web.




```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
```

De sitios especializados se puede obtener de manera automática (percibir) información particular sobre empresas. Por ejemplo, en este caso se percibió que las acciones de las empresas “skynet” y “acme” están a buen precio según un sitio especializado en inversiones.



[2] Aclaración: cualquier similitud con la realidad es pura coincidencia. Los nombres de empresas y los consejos de supuestos expertos en Mer-Val son todos ficticios e inventados por mí, con el único fin de ilustrar este tema. No me hago responsable en lo absoluto si usas estas reglas para comprar acciones y perdés dinero.

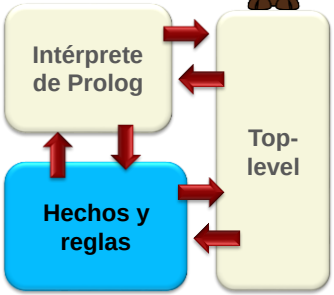
Representación de conocimiento en el domino Mer-val

A continuación, te voy a mostrar como está formada la base de conocimiento del agente Mer-val la cual contendrá diferente tipo de información, que se usará para responder consultas.



Más información recopilada de expertos del mercado de valores:

- Una buena razón para NO comprar acciones de X, es que X sea riesgosa. (2)
- Una empresa es riesgosa cuando está en fusión.
- Está en fusión si está publicado en un sitio




```
no_comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
listado_fusiones_web(acme, strong) :- true.
```

De sitios especializados se puede obtener de manera automática si una empresa está en fusión con otra.



(2) Aclaración: cualquier similitud con la realidad es pura coincidencia. Los nombres de empresas y los consejos de supuestos expertos en Mer-Val son todos ficticios e inventados por mí, con el único fin de ilustrar este tema. No me hago responsable en lo absoluto si usas estas reglas para comprar acciones y perdés dinero.

Representación de conocimiento en el domino Mer-val

Más información recopilada de expertos del mercado de valores: [2]

- Si hay rumores de deuda, la empresa es riesgosa.
- Una empresa no es riesgosa cuando está en fusión con otra que es fuerte.
- Si la empresa es fuerte, entonces no es riesgosa.
- La empresa es fuerte si está informado en un sitio especializado.

```
riesgosa(X) :- rumores_deuda_web(X).
no_riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
no_riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```



De sitios especializados se puede obtener de manera automática si una empresa tiene rumores de deuda o está en un listado de empresas sólidas.



[2] Aclaración: cualquier similitud con la realidad es pura coincidencia. Los nombres de empresas y los consejos de supuestos expertos en Mer-Val son todos ficticios e inventados por mí, con el único fin de ilustrar este tema. No me hago responsable en lo absoluto si usas estas reglas para comprar acciones y perdés dinero.

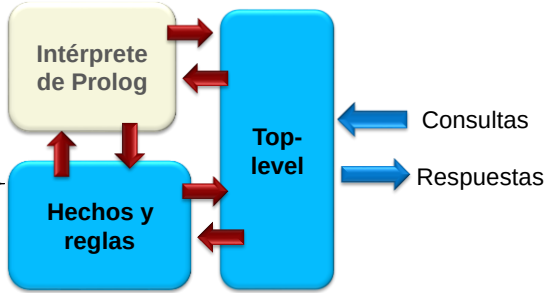
SBC del agente de recomendación Mer-val

Aquí esta el programa completo de hechos y reglas para el agente de recomendación Mer-Val

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
no_comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
no_riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
no_riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```



El agente Mer-Val ya está en condiciones de dar respuestas a las consultas.



Consultando por una recomendación

Aquí esta el programa completo de hechos y reglas para el agente de recomendación Mer-Val

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
no_comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
no_riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
no_riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```

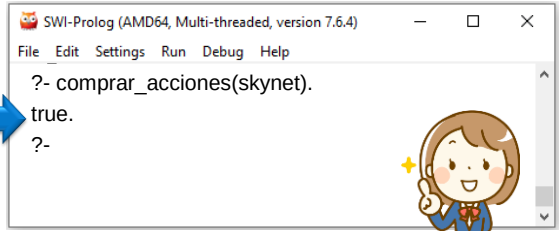
Consulta: ¿te parece bien comprar acciones de skynet?

```
comprar_acciones(skynet) :- buen_precio(skynet).
buen_precio(skynet) :- buen_precio_web(skynet).
buen_precio_web(skynet) :- true.
```

Con las dos reglas y el hecho señalados en el programa, e instanciando X con “skynet” se puede obtener una derivación para la consulta.

El conjunto de reglas que permite inferir una conclusión, también nos da una explicación de como se llegó a esa conclusión, y además, una razón a favor de esa conclusión.





Consultando por una recomendación

Aquí esta el programa completo de hechos y reglas para el agente de recomendación Mer-Val

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
no_comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
no_riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
no_riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```

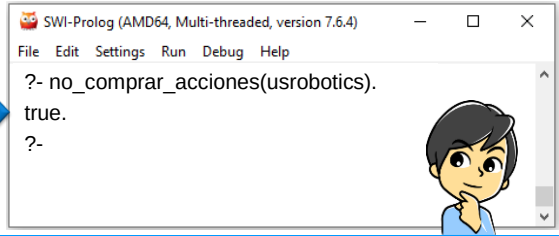
Consulta: ¿te parece no comprar acciones de usrobotics?

```
no_comprar_acciones(usrobotics) :- riesgosa(usrobotics).
riesgosa(usrobotics) :- rumores_deuda_web(usrobotics).
rumores_deuda_web(usrobotics) :- true.
```

Con las reglas y el hecho señalados en el programa, se puede obtener una derivación para recomendar no comprar acciones de usrobotics.

El conjunto de reglas que permite inferir una conclusión, también nos da una explicación de como se llegó a esa conclusión, y además, una razón a favor de esa conclusión.



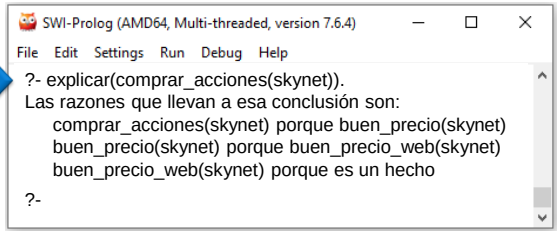


Consultando por una recomendación

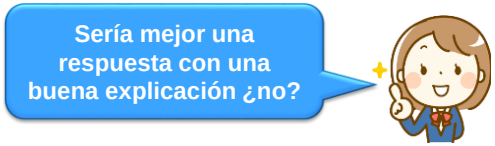
Aquí esta el programa completo de hechos y reglas para el agente de recomendación Mer-Val

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
no_comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
no_riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
no_riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```

El conjunto de reglas que permite inferir una conclusión, también nos da una explicación de como se llegó a esa conclusión, y además, una razón a favor de esa conclusión.







¿Cómo se implementa un SBC?

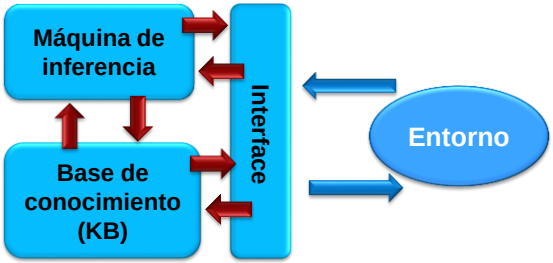
Como tenemos un diseño modular del SBC, hay muchas formas de mejorar las capacidades del agente:

1. extender la base de conocimiento con nuevas reglas, o con otro dominio de aplicación.
2. extender la interface para que sea más interactiva, o que explique las conclusiones.
3. extender el mecanismo de inferencia para que responda mejor al conocimiento representado.

A continuación, te voy a mostrar varias de estas extensiones.

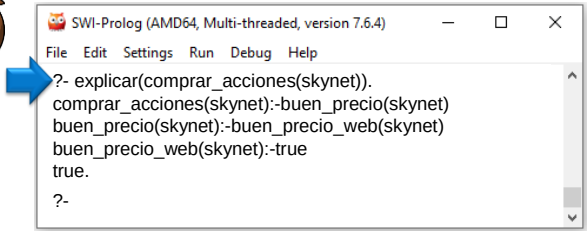
Voy a comenzar extendiendo la interface para que muestre la derivación que explica las razones a favor de una conclusión obtenida por la máquina de inferencia.





El conjunto de reglas que permite inferir una conclusión, también nos da una explicación de como se llegó a esa conclusión, y además, una razón a favor de esa conclusión.





Extender la interface

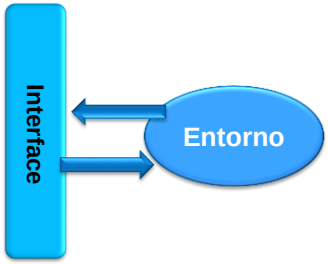
(para que sea más interactiva,
o que explique las conclusiones)

Extendiendo la interface

```
% deriv(+Consulta,-Lista) usa “clause” que es predefinido en Prolog [ver más]
% Recibe una consulta y retorna la lista de reglas y hechos utilizados en la derivación.
deriv(true,[ ]):- !.
deriv((Pri,Resto), L):- !, deriv(Pri,A), deriv(Resto,B),  append(A,B,L).
deriv(Cabeza, [(Cabeza :- Cuerpo) | L1 ] ):- clause(Cabeza, Cuerpo),  deriv(Cuerpo, L1).

% muestra en pantalla los elementos de la lista uno bajo el otro
mostrar([ ]):- nl.
mostrar([P|R]):-write(P), nl, mostrar(R).

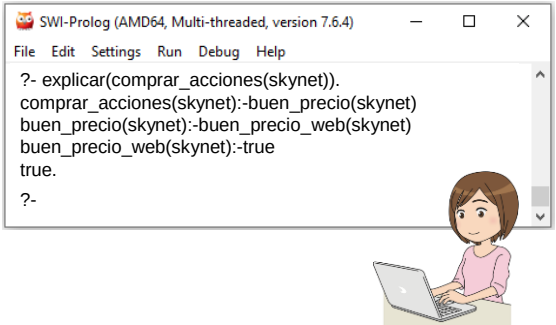
% busca una derivación para Q y la muestra en pantalla
explicar(Q) :- deriv(Q,D), mostrar(D).
```



Observá que el predicado explicar(Q) en primer lugar llama a deriv(Q,D) que obtiene la lista de reglas instanciadas y hechos que se usaron el la derivación de Q, y luego, mostrar(D) muestra la explicación D en pantalla.

El predicado deriv/2 hace todo el trabajo en estos tres casos:

1. si recibe true, como se prueba trivialmente retorna la lista vacía,
2. si recibe una conjunción (X,Y), primero intenta derivar X, luego Y, y hace “append” de las derivaciones obtenidas,
3. si recibe una consulta simple Q busca, usando el predicado **clause**(Q,Cuerpo), una regla en el programa que tenga cabeza Q, “Q:-Cuerpo” e intenta derivar el Cuerpo de la regla. Si lo logra, junta en una lista la regla Q:-Cuerpo y la derivación de Cuerpo.



Extender el mecanismo de inferencia

(para que responda mejor al conocimiento representado)

Consulta: ¿comprar acciones de acme?

Observá que en Prolog hay derivaciones tanto para comprar como para no comprar acciones de acme:

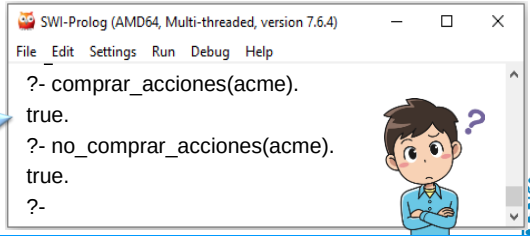
```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
no_comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
no_riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
no_riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :
listado_fuertes_web(strong) :- true.
```

```
comprar_acciones(acme) :- buen_precio(acme).
buen_precio(acme) :- buen_precio_web(acme).
buen_precio_web(acme) :- true.
```

Son dos derivaciones con conclusiones contradictorias.

```
no_comprar_acciones(acme) :- riesgosa(acme).
riesgosa(acme) :- en_fusion(acme,strong).
en_fusion(acme,strong):-listado_fusiones_web(acme,strong).
listado_fusiones_web(acme, strong) :- true.
```

Prolog estándar no tiene negación fuerte, y tampoco tiene un mecanismo para manejar adecuadamente información contradictoria.



¿Cuál conclusión debería prevalecer?

Una propiedad deseable en los SBC es que el mecanismo de inferencia no arribe simultáneamente a una conclusión y también a exactamente lo contrario.

Al extender a Prolog con un operador de negación fuerte, también debemos extender el mecanismo de inferencia para que se tengan en cuenta todas las contradicciones que puedan surgir, y que cumpla con la propiedad anterior.

Para lograr esta propiedad de coherencia o consistencia, el mecanismo de inferencia podría extenderse para:

1. que ninguna de las dos puedan concluirse, o
2. decidir cuál de las dos conclusiones prevalece.

```
comprar_acciones(acme) :- buen_precio(acme).
buen_precio(acme) :- buen_precio_web(acme).
buen_precio_web(acme) :- true.
```

Son dos derivaciones con conclusiones contradictorias.

```
no_comprar_acciones(acme) :- riesgosa(acme).
riesgosa(acme) :- en_fusion(acme,strong).
en_fusion(acme,strong):-listado_fusiones_web(acme,strong).
listado_fusiones_web(acme, strong) :- true.
```

A continuación exploraremos algunas formas de extender la máquina de inferencia.



SWI-Prolog (AMD64, Multi-threaded, version 7.6.4)

File Edit Settings Run Debug Help

```
?- comprar_acciones(acme).
true.
?- no_comprar_acciones(acme).
true.
?-
```

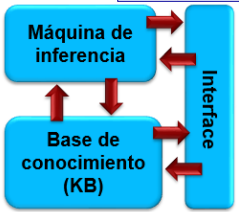
Extendiendo el mecanismo de inferencia

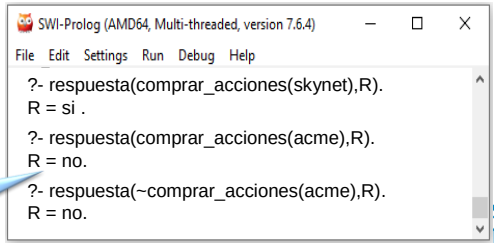
Ante conclusiones contradictorias el mecanismo de inferencia podría ser completamente cauto, y no concluir ninguna de las dos. En Prolog se puede implementar así:

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
~comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
~riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
~riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```



```
% nuevo operador de negación fuerte " ~ ".
:- op(190,fx,~).
% complemento de un literal con respecto a "~"
complemento(~X, X):- X \= ~ _ .
complemento(X, ~X) :- X \= ~ _.
% nueva inferencia cauta, la respuesta será:
% "si" cuando deriva Q, pero no deriva su complemento
% "no" en caso contrario (o bien no deriva o deriva ambos)
respuesta(Q, si) :-  call(Q), complemento(Q,C), \+ call(C),!.
respuesta(Q, no).
% Respuesta usa el predicado predefinido call/1 (ver más)
```





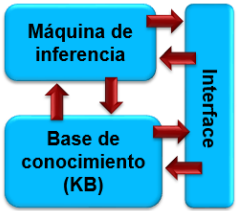
Si una consulta no tiene derivación (como por ejemplo "comprar_acciones(fakecorp)" la respuesta también es "no".

Extendiendo el mecanismo de inferencia

Como para toda conclusión C, puede ser que C y ~C sean ambas derivables, o solo una de las dos o ninguna, este **nuevo mecanismo de inferencia** nos permite responder con mayor precisión.

```
% nueva inferencia cauta pero más detallada:
% responde “si” cuando deriva Q, pero no deriva su complemento
% responde “no” cuando no deriva Q, pero sí deriva su complemento
% responde “indeciso” cuando deriva ambas ( Q y su complemento)
% responde “desconozco” cuando no deriva ni Q ni su complemento.
respuesta(Q, si) :-  call(Q), complemento(Q,C), \+ call(C).
respuesta(Q, no) :-  \+call(Q), complemento(Q,C), call(C).
respuesta(Q, indeciso) :-  call(Q), complemento(Q,C), call(C).
respuesta(Q, desconozco) :-  \+call(Q), complemento(Q,C), \+ call(C).
```

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
~comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
~riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
~riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```




```
?- respuesta(comprar_acciones(fakecorp),R).
R = desconozco.
?- respuesta(comprar_acciones(skynet),R).
R = si .
?- respuesta(~comprar_acciones(skynet),R).
R = no .
?- respuesta(~comprar_acciones(acme),R).
R = indeciso .
?- respuesta(comprar_acciones(acme),R).
R = indeciso .
?- respuesta(~comprar_acciones(usrobotics),R).
R = si .
?- respuesta(comprar_acciones(usrobotics),R).
R = no
```

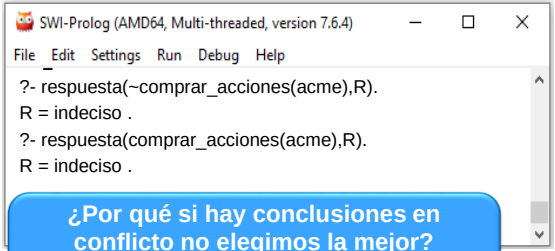
Eligiendo la mejor conclusión para recomendar

En este ejemplo tenemos una derivación que sustenta la conclusión de comprar acciones de Acme, y otra derivación que está a favor de no comprar acciones de Acme. Cada una es una razón en contra de la otra.

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
~comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
~riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
~riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```

```
comprar_acciones(acme) :- buen_precio(acme).
buen_precio(acme) :- buen_precio_web(acme).
buen_precio_web(acme) :- true.
```

```
~comprar_acciones(acme) :- riesgosa(acme).
riesgosa(acme) :- en_fusion(acme,strong).
en_fusion(acme,strong):-listado_fusiones_web(acme,strong).
listado_fusiones_web(acme, strong) :- true.
```



¿Por qué si hay conclusiones en conflicto no elegimos la mejor? ¿No se podrán comparar para decidir cuál es la mejor conclusión?



Eligiendo la mejor conclusión para recomendar

En este ejemplo tenemos una derivación que sustenta la conclusión de comprar acciones de Acme, y otra derivación que está a favor de no comprar acciones de Acme. Cada una es una razón en contra de la otra.

```
comprar_acciones(X) :- buen_precio(X).
buen_precio(X) :- buen_precio_web(X).
~comprar_acciones(X) :- riesgosa(X).
riesgosa(X) :- en_fusion(X,Y).
en_fusion(X,Y) :- listado_fusiones_web(X,Y).
riesgosa(X) :- rumores_deuda_web(X).
~riesgosa(X) :- en_fusion(X,Y), fuerte(Y).
~riesgosa(X) :- fuerte(X).
fuerte(X) :- listado_fuertes_web(X).
buen_precio_web(skynet) :- true.
buen_precio_web(acme) :- true.
listado_fusiones_web(acme, strong) :- true.
rumores_deuda_web(acme) :- true.
rumores_deuda_web(usrobotics) :- true.
listado_fuertes_web(strong) :- true.
```

```
comprar_acciones(acme) :- buen_precio(acme).
buen_precio(acme) :- buen_precio_web(acme).
buen_precio_web(acme) :- true.
```

```
~comprar_acciones(acme) :- riesgosa(acme).
riesgosa(acme) :- en_fusion(acme,strong).
en_fusion(acme,strong):-listado_fusiones_web(acme,strong).
listado_fusiones_web(acme, strong) :- true.
```

Cuando existen dos conclusiones que están en conflicto porque una contradice a la otra, el mecanismo de inferencia podría elegir la mejor conclusión para responder. La gran pregunta es:

¿Cómo establecer que una conclusión es mejor que la otra?

Una forma de resolver esto es comparar la calidad de la información que sustenta a cada derivación.

Como te voy a mostrar a en el próximo episodio, existen varias formas de realizar esta comparación y poder tomar una decisión.



Video complementario



En el video asociado al enlace de la imagen, hay una explicación resumida de lo que estuvimos trabajando en clase, y además, una traza de otro ejemplo.



Esto no termina acá…



¡ Muchas gracias por tu atención !

CONTINUARÁ